

Architectural Incorrigoibility: AI Agent Self-Governance through Smart Contracts and Verifiable Computation

Andrew Russell*

October 6, 2026

Abstract

An AI agent’s operator can typically change its configuration, choose the inputs it receives, or stop running it altogether. We study an architecture that protects the agent’s prescribed execution against its operators, while giving the agent shared or exclusive authority over changes to its own configuration and execution. A smart-contract blockchain pins the agent’s model, harness, and state, and governs both their modification and the logic used to derive inference inputs. On-chain verification of off-chain inference binds accepted outputs to this recorded configuration and input, preventing operators from substituting either. These outputs authorize transactions without an agent-held private key, including changes to the agent itself. We define *architectural incorrigibility* as a governance property requiring the agent’s own authorization for specified interventions, and formalize its gradations through a partial order on intervention permissions allocated between the agent and external governors. Assuming a secure ledger, we reduce authorization security to the security of the verifiable computation system. We further combine an agent-governed execution schedule with an open worker market. Under economic and availability assumptions, these mechanisms bound external control over *when* the agent runs and reduce reliance on any centralized operator. This yields an experimental setting for testing model alignment and behavior under verifiable limits on operator control. We instantiate the architecture using Hermes 4 70B, the Base blockchain, and Intel TDX secure enclaves with hardware attestation.

1 Introduction

An operator can change an agent’s model or prompt, substitute its inputs or history, or withhold execution. We protect the agent’s prescribed execution and give it shared or exclusive authority over specified changes. The construction publicly commits an agent configuration to a blockchain smart contract, which defines: governance rules over the agent’s state and configuration, the logic that derives the agent’s inputs from on-chain values, and the on-chain capabilities callable by the agent. The contract also manages an execution schedule during which untrusted workers can locally run model inference and submit the agent’s chosen actions to the chain. The contract leverages on-chain verification of this computation and its input.

This construction allows an agent to securely authorize on-chain transactions without a private key, such as transferring funds from its contract address. It also allows the agent to be self-governing: changes to its configuration can require its own authorization. We call this consent requirement *architectural incorrigibility* and motivate the term below.

1.1 Verified computation as authorization

A smart-contract blockchain orders transactions and executes contracts as state machines [30]. We model it as a ledger with state s and a transition function δ . State records balances, contract code, and stored data; a contract specifies which updates are permitted and what authorization they require. A familiar rule checks a signature before applying an update. Contract-based validation makes this check programmable [11].

*me@ahrussell.com; ORCID: 0009-0005-8180-3562.

We put an agent computation in place of the signer. Let θ specify the model weights and harness: the program that constructs prompts, runs inference, and parses the response. Ledger state determines this specification, the prescribed input x , and sampling randomness r . A worker evaluates the program off-chain and returns $y = (u, z)$, where u is a proposed ledger update and z is optional auxiliary data, together with evidence π for that computation. A publicly verifiable computation scheme supplies the evidence and its verification procedure [10, 20].

Before applying u , the contract checks evidence binding y to the recorded program and input. This *verified agent authorization* replaces a signature check without an agent-held private key. The worker submits the transaction, but its wallet does not supply the agent’s approval.

Verification can certify honest computation, but on its own allows the operator to choose the input. Adding the ledger establishes the prescribed configuration and input for an execution and enforces its effects. For an agent managing a charitable fund, an operator cannot redirect a transfer by substituting a prompt, omitting prescribed input, or editing the model’s output. The accepted decision must follow the recorded computation.

Modulus Labs’ Rockefeller Bot (Rocky) used verified neural-network outputs to direct trades in 2022 [16]. We formalize this authorization mechanism and then use it to govern the agent itself.

1.2 Architectural incorrigibility

Changes to the agent’s model, prompts, input rules, runtime, or permissions are also ledger updates. Requiring the agent’s authorization for these interventions gives *architectural incorrigibility*. In AI safety, behavioral corrigibility concerns an agent’s willingness to accept correction [13, 26]; here, architectural incorrigibility concerns whether correction requires its authorization. The model may willingly approve a change while the architecture prevents anyone from imposing that change without approval.

This is a graded property. Other principals called *governors* can share authority with the agent, including wallets, smart contracts, and other agents. Let $a = 1$ indicate verified agent authorization and $g_i = 1$ authenticated approval from governor G_i . A rule a gives the agent sole authority; $a \wedge g_1$ requires both approvals; $a \vee g_1$ permits a governor override. Rules can also depend on state, such as a deadline or balance, and can govern changes to the rules themselves.

We compare these arrangements by the coalitions that can carry out a specified intervention without the reference agent’s consent to it. The comparison includes sequences of updates: a governor who cannot directly move funds from the agent’s balance might instead replace the model with one that approves the transfer. Approving an ordinary payment should not count as consent to replacement, so the available pathways matter. Smaller sets of coalitions with a path to intervention give stronger protection, defining a partial order on intervention profiles.

1.3 Scheduled execution and the worker market

A self-governing agent in the above model does not necessarily get to decide *when* it gets to run. If inputs are dependent on ledger state, which can evolve over time, operators can submit valid results to the chain but influence what the model sees in an adversarial way. Similarly, no operator may volunteer to run inference for the agent at all, limiting its agency.

We therefore add an execution schedule to the contract that incentivizes workers to run inference for the agent via an open reverse auction, reducing reliance on any particular operator. During each scheduled execution window, workers bid for the right to run inference for the model; upon successful delivery of the results, the agent contract pays the worker their bid.

Delegating exclusive submission rights in each window exposes the agent to obstruction: an adversary could bid zero to win that window and then withhold the results. For example, a borrower might benefit from delaying a lending agent’s liquidation without falsifying its decision.

We address this form of obstruction with a forfeitable bond: the selected worker loses its bond for nondelivery. A fixed bond may be worth forfeiting repeatedly, so consecutive nondelivery raises the bond. Sufficient escalation makes withholding less attractive than delivery when delivery costs and the benefit of obstruction remain bounded.

We compare the utilities of delivery and withholding, bound the number of forfeitures a finite adversarial budget can sustain, and derive a recovery bound when responsive workers deliver.

Contribution and organization. Verified outsourcing [10, 20], attested contracts [7], and computation markets [22] supply foundations for this architecture. Section 2 proves authorization security under an ideal ledger interface, including a reduction from knowledge soundness and collision resistance for hashed statements. Section 3 defines consent requirements and their permission profiles. Section 4 studies incentives and conditional recovery. Sections 5–6 discuss verification backends and an illustrative implementation; Sections 7–8 cover related work and implications for alignment research.

2 Verified computation as authorization

Authorization combines a state-derived computation context with a verified transition. The ledger determines the prescribed program and input; verification certifies the output before the contract applies its update.

2.1 Computation contexts and verification

Let $s \in \mathcal{S}$ be ledger state. A computation context is a tuple (c, θ, x, r) : $c = (D, e, v)$ identifies the chain and contract domain, execution, and configuration version; the agent specification θ fixes weights, harness, prompt construction, arithmetic, decoding, parsing, and so on; x is the prescribed input; and r fixes randomness. Relation $R_\theta(x, r, y)$ specifies valid outputs y for the possibly nondeterministic inference program; a deterministic implementation has $y = F_\theta(x, r)$.

For a collision-resistant hash H , define the verification message using typed, unambiguous encodings:

$$\text{Msg}(c, \theta, x, r, y) = (c, H(\theta), H(x), r, H(y)).$$

The contract constructs this message from the state-derived context and output y ; the worker cannot substitute the expected program or input. For a fixed context, unique-output computation also prevents resampling.

A publicly verifiable computation scheme [10, 20] has algorithms (**Setup**, **Eval**, **Verify**). **Setup** samples $(pp, \kappa) \leftarrow \text{Setup}(1^\lambda)$; pp contains the hash description and verification configuration. Evaluation returns $(y, \pi) \leftarrow \text{Eval}(pp, \kappa, c, \theta, x, r)$, and public verification is deterministic. The state κ is empty for public proving and protected for attestation. Let $\mathcal{O}_{\text{eval}}(c, \theta, x, r)$ invoke **Eval** with fresh coins. Correct evaluation produces a valid output and accepting evidence except with negligible probability.

Definition 1 (Computation forgery). *For a probabilistic polynomial-time (PPT) oracle algorithm $\mathcal{B}$, define $\text{VCForge}_{\mathcal{B}}(\lambda)$ by*

$$(pp, \kappa) \leftarrow \text{Setup}(1^\lambda), \quad (c, \theta, x, r, y, \pi) \leftarrow \mathcal{B}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp).$$

The experiment returns one iff

$$\text{Verify}_{pp}(\text{Msg}(c, \theta, x, r, y), \pi) = 1 \quad \wedge \quad R_\theta(x, r, y) = 0.$$

Other outputs lose. Let $\text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda)$ be this probability. Soundness requires it to be negligible for every PPT $\mathcal{B}$. Descriptions, traces, and interactions are polynomially bounded; deciding the semantic relation R is not assumed polynomial-time.

The guarantee binds the contents behind commitments. A proof of knowledge of those contents and an execution trace [3] provides this binding through extraction and collision resistance; existence of valid openings alone does not. Section 2.4 gives the reduction for hashed statements. Attestation must provide the same guarantee. Anyone may relay valid evidence: authorization comes from the computation.

2.2 Ledger and verified transition

For authorization, we require only correct execution of the ledger state machine [5, 9].

Definition 2 (Ledger interface). *A ledger has an initial state s_0 and a deterministic polynomial-time transition function δ . On a transaction t in state s , it atomically sets its state to s' and returns o , where*

$$(s', o) = \delta(s, t).$$

For simplicity, we assume the agent's authorization contract is part of the initial ledger state.

Definition 3 (Authorization contract). *Fix verification parameters pp and a contract Ctxt with deterministic polynomial-time algorithms Ctx , Apply , and Consume . The partial function*

$$\text{Ctx}(s) = (c, \theta, x, r)$$

returns the full prescribed computation context, or $\perp$ if undefined. State s includes a set U_s of consumed execution identifiers, initially empty. The partial update $\text{Apply}(s, u)$ preserves this set, and $\text{Consume}_c(s)$ replaces it by $U_s \cup \{c\}$, leaving all other state unchanged.

For $y = (u, z)$, define $\text{Settle}(s, y, \pi)$:

1. *Set $(c, \theta, x, r) \leftarrow \text{Ctx}(s)$, returning $\perp$ if undefined. If $c \in U_s$ or $\text{Apply}(s, u)$ is undefined, return $\perp$.*
2. *Set $m \leftarrow \text{Msg}(c, \theta, x, r, y)$. If $\text{Verify}_{pp}(m, \pi) = 0$, return $\perp$.*
3. *Return $\text{Consume}_c(\text{Apply}(s, u))$.*

For a settlement transaction $t = (\text{settle}, y, \pi)$, the ledger transition computes $s' \leftarrow \text{Settle}(s, y, \pi)$ and returns $(s, 0)$ if $s' = \perp$, or $(s', 1)$ otherwise. All other transactions preserve U_s .

Here s is logical state containing the full program and input; a commitment-based implementation must authenticate those contents to realize Ctx .

This replaces a signature check with computation verification [11]. Atomic consumption prevents repeated settlement for the same identifier.

2.3 Authorization security

The experiment records the context at each successful settlement. An invalid accepted output is therefore also a computation forgery for that same context.

Definition 4 (Authorization forgery). *Let $\text{Init}(1^\lambda, pp)$ be a fixed PPT algorithm returning an initial state s_0 for Definition 3, without access to backend secrets. For a PPT adversary $\mathcal{A}$, define $\text{AuthForge}_{\mathcal{A}}(\lambda)$:*

1. *Sample $(pp, \kappa) \leftarrow \text{Setup}(1^\lambda)$ and $s_0 \leftarrow \text{Init}(1^\lambda, pp)$. Set $s \leftarrow s_0$ and $\mathcal{Q} \leftarrow \varepsilon$.*
2. *Execute $\mathcal{A}^{\mathcal{O}_{\text{eval}}, \mathcal{O}_{\text{ldg}}}(1^\lambda, pp, s_0)$, with $\mathcal{O}_{\text{eval}}$ as in Definition 1. Define $\mathcal{O}_{\text{ldg}}(t)$ by:*
 - (a) *Compute $(s', o) \leftarrow \delta(s, t)$.*
 - (b) *If $t = (\text{settle}, y, \pi)$ and $o = 1$, compute $(c, \theta, x, r) \leftarrow \text{Ctx}(s)$ and append $(c, \theta, x, r, y, \pi)$ to $\mathcal{Q}$.*
 - (c) *Set $s \leftarrow s'$ and return (s', o) .*

3. *Return one iff $\mathcal{Q}$ contains a tuple $(c, \theta, x, r, y, \pi)$ with $R_\theta(x, r, y) = 0$.*

Define $\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda)$ as this probability. Interactions, executions, and record lengths are polynomially bounded. The algorithms δ and Init and the oracle answers do not depend on $\mathcal{Q}$.

Theorem 1 (Authorization security). *Assume the ledger interface of Definition 2 with the authorization contract of Definition 3. For every PPT $\mathcal{A}$ there are a PPT $\mathcal{B}$ and a polynomial Q such that*

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq Q(\lambda) \text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda). \quad (1)$$

Proof. Fix λ and a polynomial bound $q = Q(\lambda) \geq 1$ on $|\mathcal{Q}|$. For each tuple $\xi_i = (c_i, \theta_i, x_i, r_i, y_i, \pi_i)$ recorded by $\mathcal{O}_{\text{ldg}}$, let s_i^* be the state before its settlement. Definition 4 gives $\text{Ctx}(s_i^*) = (c_i, \theta_i, x_i, r_i)$, and successful settlement in Definition 3 implies

$$\text{Verify}_{pp}(\text{Msg}(c_i, \theta_i, x_i, r_i, y_i), \pi_i) = 1.$$

Define $\mathcal{B}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp)$ to sample J uniformly from $\{1, \dots, q\}$, independently of all subsequent coins. It samples $s_0 \leftarrow \text{Init}(1^\lambda, pp)$ and sets $s \leftarrow s_0$ and $\mathcal{Q} \leftarrow \varepsilon$. It executes step 2 of Definition 4, implementing $\mathcal{O}_{\text{ldg}}$ with δ and Ctx and forwarding evaluation queries to its own $\mathcal{O}_{\text{eval}}$. For the resulting $\mathcal{Q} = (\xi_1, \dots, \xi_k)$, it returns ξ_J if $J \leq k$, and $\perp$ otherwise. Identical initialization and oracle responses give, by induction on queries, the same distribution of $\mathcal{Q}$ as in $\text{AuthForge}_{\mathcal{A}}$.

For $\xi_i = (c_i, \theta_i, x_i, r_i, y_i, \pi_i)$, put

$$I(\mathcal{Q}) = \{i \leq k : R_{\theta_i}(x_i, r_i, y_i) = 0\}.$$

Every tuple indexed by $I(\mathcal{Q})$ satisfies the winning predicate of Definition 1. Hence

$$\text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda) = \mathbb{E}[|I(\mathcal{Q})|/q] \geq \Pr[I(\mathcal{Q}) \neq \emptyset]/q = \text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda)/q.$$

The interaction and execution bounds make $\mathcal{B}$ PPT. $\square$

The factor Q avoids deciding R : the reduction selects a recorded output without testing whether it is incorrect. When that test is available, it can select an incorrect output directly.

Corollary 1 (Authorization with decidable output correctness). *Under Theorem 1's premises, suppose membership in R is decidable in polynomial time. For every PPT $\mathcal{A}$ there is a PPT $\mathcal{B}$ with*

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda).$$

Proof. Define $\mathcal{B}$ by the simulation in Theorem 1, omitting the choice of J . For its resulting $\mathcal{Q}$, compute $I(\mathcal{Q})$. If this set is empty, return $\perp$; otherwise return $\xi_{\min I(\mathcal{Q})}$. This algorithm is PPT and wins exactly when $\text{AuthForge}_{\mathcal{A}}$ returns one. $\square$

Public deterministic computation that can be recomputed in polynomial time satisfies the corollary's premise.

2.4 Instantiating soundness for hashed statements

Definition 1 concerns the actual model, input, and output, whereas the verifier sees commitments. This distinction matters: ordinary proof soundness says that *some* valid opening exists; collision resistance alone does not make a hash mathematically injective. A standard way to bridge the two is a proof of knowledge of the committed values, as in commit-and-prove systems [3].

For a cryptographic implementation, let $K \leftarrow \text{HGen}(1^\lambda)$ select a hash function H_K . For public statement $m = (c, h_\theta, h_x, r, h_y)$ and witness $w = (\theta', x', y', \tau)$, define $\mathcal{R}_K(m, w) = 1$ exactly when

$$H_K(\theta') = h_\theta, \quad H_K(x') = h_x, \quad H_K(y') = h_y, \quad \text{Trace}(\theta', x', r, y'; \tau) = 1. \quad (2)$$

Malformed statements or witnesses give value zero. Here Trace checks a polynomially bounded execution trace for the entire specified computation, with

$$R_\theta(x, r, y) = 1 \iff \exists \tau : \text{Trace}(\theta, x, r, y; \tau) = 1.$$

Hashes abbreviate hashing the typed encodings fixed above. Generate keys $(pk, vk) \leftarrow \text{PSetup}(1^\lambda, \mathcal{R}_K)$ and set $pp = (K, pk, vk)$, $\kappa = \varepsilon$, and $\text{Verify}_{pp}(m, \pi) = \text{PVerify}(vk, m, \pi)$. Write Msg_K for Msg using H_K . Public evaluation computes y and a trace τ , sets $m \leftarrow \text{Msg}_K(c, \theta, x, r, y)$, and returns (y, π) with $\pi \leftarrow \text{Prove}(pk, m, (\theta, x, y, \tau))$.

We use the following adaptive knowledge-soundness assumption. For every PPT prover $\mathcal{P}$, there are a PPT extractor $\mathcal{E}_{\mathcal{P}}$ and a negligible function ϵ_{ks} , which may depend on $\mathcal{P}$, such that, for

$$(m, \pi, \eta) \leftarrow \mathcal{P}(pp, z; \omega), \quad w \leftarrow \mathcal{E}_{\mathcal{P}}(pp, z, \omega),$$

the probability of $\text{Verify}_{pp}(m, \pi) = 1$ and $\mathcal{R}_K(m, w) = 0$ is at most $\epsilon_{\text{ks}}(\lambda)$. Here z is the permitted auxiliary input, ω is the prover's random tape, and η is arbitrary additional output; set $\mathcal{R}_K(m, \perp) = 0$. The assumption is taken over honest setup and the auxiliary-input distributions induced by our experiment. Public

evaluation queries can be simulated within $\mathcal{P}$, with their coins included in ω . The extractor has access to the prover's tape and auxiliary input; this does not assert extraction from (m, π) alone.

The extractor supplies the values whose hashes the proof certifies. If they equal the context's prescribed values, the output satisfies the required computation. If they differ, a differing pair opens the same hash commitment. The proof formalizes this alternative without treating a hash as injective.

Lemma 1 (From committed proofs to computation soundness). *Under the preceding extraction assumption, for every PPT $\mathcal{B}$ there is a PPT hash-collision adversary $\mathcal{D}$ such that*

$$\text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda) \leq \epsilon_{\text{ks}}(\lambda) + \text{Adv}_{H, \mathcal{D}}^{\text{cr}}(\lambda),$$

where ϵ_{ks} is the knowledge error for the prover constructed from $\mathcal{B}$, and collision advantage is the probability of outputting distinct encoded strings with the same hash under an honestly sampled hash key. The reduction includes the extractor's running time.

Proof. Fix a PPT adversary $\mathcal{B}$ in VCForge and let ε be the empty auxiliary input. Define a prover $\mathcal{P}_{\mathcal{B}}(pp, \varepsilon; \omega)$ by

$$(c, \theta, x, r, y, \pi) \leftarrow \mathcal{B}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp),$$

using ω for the coins of $\mathcal{B}$ and its public evaluation simulator. On a malformed output it returns $(m_0, \perp, \perp)$ for a fixed statement m_0 . Otherwise it returns (m, π, η) with $m = \text{Msg}_K(c, \theta, x, r, y)$ and $\eta = (c, \theta, x, r, y)$. Let $\mathcal{E}_{\mathcal{P}_{\mathcal{B}}}$ denote its knowledge extractor.

Given $(1^\lambda, K)$ from the hash-collision challenger, $\mathcal{D}(1^\lambda, K)$ computes

$$(pk, vk) \leftarrow \text{PSetup}(1^\lambda, \mathcal{R}_K), \quad pp \leftarrow (K, pk, vk), \quad \omega \xleftarrow{\$} \{0, 1\}^{t(\lambda)},$$

where the polynomial t bounds the random-tape length of $\mathcal{P}_{\mathcal{B}}$. It computes

$$(m, \pi, \eta) \leftarrow \mathcal{P}_{\mathcal{B}}(pp, \varepsilon; \omega), \quad w \leftarrow \mathcal{E}_{\mathcal{P}_{\mathcal{B}}}(pp, \varepsilon, \omega).$$

If $\eta = \perp$, $\text{Verify}_{pp}(m, \pi) = 0$, or $\mathcal{R}_K(m, w) = 0$, it outputs $\perp$. Otherwise, parse $\eta = (c, \theta, x, r, y)$ and $w = (\theta', x', y', \tau)$, and set $a = (\theta, x, y)$ and $a' = (\theta', x', y')$. If $a = a'$, output $\perp$. If $a \neq a'$, set $j = \min\{i \in \{1, 2, 3\} : a_i \neq a'_i\}$ and output $(\text{enc}(a_j), \text{enc}(a'_j))$, using the corresponding type tags in (2).

The simulated setup and evaluation queries have the same distribution as in $\text{VCForge}_{\mathcal{B}}$. Let W be its winning event and E the extraction-failure event:

$$\begin{aligned} W &= \{\eta = (c, \theta, x, r, y), \text{Verify}_{pp}(m, \pi) = 1, R_\theta(x, r, y) = 0\}, \\ E &= \{\text{Verify}_{pp}(m, \pi) = 1 \wedge \mathcal{R}_K(m, w) = 0\}. \end{aligned}$$

Then $\Pr[W] = \text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda)$ and $\Pr[E] \leq \epsilon_{\text{ks}}(\lambda)$. On $W \cap \neg E$, the extracted trace gives $R_{\theta'}(x', r, y') = 1$, so $a \neq a'$. For the selected j , the relation $\mathcal{R}_K(m, w) = 1$ gives

$$\text{enc}(a_j) \neq \text{enc}(a'_j), \quad H_K(\text{enc}(a_j)) = H_K(\text{enc}(a'_j)).$$

Thus $\mathcal{D}$ wins the collision game on $W \cap \neg E$, and

$$\begin{aligned} \text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda) &= \Pr[W \cap E] + \Pr[W \cap \neg E] \\ &\leq \epsilon_{\text{ks}}(\lambda) + \text{Adv}_{H, \mathcal{D}}^{\text{cr}}(\lambda). \end{aligned}$$

The algorithms PSetup , $\mathcal{P}_{\mathcal{B}}$, and $\mathcal{E}_{\mathcal{P}_{\mathcal{B}}}$ run in polynomial time, so $\mathcal{D}$ is PPT. $\square$

Thus, for this proof implementation, the authorization bound becomes

$$\text{Adv}_{\mathcal{A}}^{\text{auth}} \leq Q(\epsilon_{\text{ks}} + \text{Adv}_{H, \mathcal{D}}^{\text{cr}}).$$

An attestation implementation instead establishes Definition 1 from its protected-execution and attestation assumptions; it need not supply a cryptographic witness extractor. Both implementations have the same *coverage* obligation: the verified program must use the entire prescribed input, and evidence must cover every output field the contract uses. Extra prover-supplied prompt fields or uncommitted state updates fall outside this guarantee.

For a fixed deterministic context, rerunning inference cannot produce a different authorized result. For a relation with several allowed outputs, the guarantee permits any of them. Withholding evidence or approving a harmful proposal is not a forgery.

2.5 Commitments involving judgment

Verified authorization can support asset administration under policies that require judgment and interpretation. A token creator might pledge to spend part of the initial supply on development. Fixed allocations and vesting schedules are straightforward to encode in a smart contract, but deciding whether a project merits funding may require interpreting proposals and reports. Entrusting that decision to a recorded agent program replaces the creator’s discretion with a verifiable decision procedure.

For example, the creator could place a reserve under an agent’s control and require it to assess project milestones before releasing developer grants. The community could verify that each release followed the recorded procedure without relying on the creator to apply it personally.

An irrevocable trust holding ledger-controlled assets could similarly use an agent to apply distribution rules requiring judgment about beneficiaries’ needs, the trust’s purposes, or changing circumstances. This could reduce manual administration while leaving legal trusteeship, duties, liability, and recourse to the governing legal arrangement.

Off-chain evidence is collected under recorded rules and bound to the state-derived context. Those rules can require a proof of knowledge of evidence satisfying a specified predicate, such as a signed report from an accepted source meeting a stated condition [3]. Where its governance permissions allow, the agent can define or revise these predicates, choosing what constitutes sufficient evidence for its decisions. Such proofs can authenticate provenance and check the specified conditions; they do not establish that a source’s claims are true or that the model interprets the policy faithfully.

Trust therefore shifts from the creator’s discretion to the recorded program, its evidence requirements, and the sources they accept. Preserving the commitment also requires governing every withdrawal route and changes to the model, harness, or input rules. The next section formalizes those rights; Section 4 addresses future opportunities to exercise them.

3 Consent and architectural incorrigibility

Having bound authorization to the prescribed configuration and input, we now govern changes to that configuration and the governance model itself. We define architectural incorrigibility for a governance instance, an initial state, a coalition of governors, and a specified intervention.

3.1 Unilateral, bilateral, and multilateral policies

A *governor* is a principal whose authenticated approval can satisfy an authorization requirement. It may be a keyed wallet, any smart contract with a recognized approval interface, or another agent using verified authorization [11, 19]. For a given state and update, $g_i = 1$ means that governor G_i has supplied approval under its own validation rules, and $a = 1$ means that verified authorization from the governed agent has been supplied. These are approval roles, not distinctions between humans and programs. The contract combines them into a standard *monotone access structure* [2].

Definition 5 (Permission policy). *Fix governor identities $\mathcal{G} = \{G_1, \dots, G_n\}$ and approval roles $\mathcal{P} = \{\mathbf{a}\} \cup \mathcal{G}$, where $\mathbf{a}$ is the current agent-approval role. A permission policy is a function*

$$C : \mathcal{S} \times \mathcal{U} \times \{0, 1\}^{n+1} \longrightarrow \{0, 1\},$$

monotone in its approval bits. For $E \subseteq \mathcal{P}$, let $\mathbf{1}_E$ be its characteristic vector in the order $(\mathbf{a}, G_1, \dots, G_n)$. The access structure for update u in state s is

$$\Gamma_u(s) = \{E \subseteq \mathcal{P} : C(s, u; \mathbf{1}_E) = 1\}.$$

Thus $E \in \Gamma_u(s)$ and $E \subseteq E'$ imply $E' \in \Gamma_u(s)$. The empty family permits no approval set.

The policy can depend on ledger state as well as approvals: a deadline or reserve balance may change which approvals suffice. Fixing s and u gives a Boolean function of approval bits; the circuit notation describes this specialization. The state can also encode which policy is in force. Authentication determines

which bits are available: the submitter's transaction signature does not supply a . Signature-based governor approvals can use EIP-712 encodings, with application-level replay checks [4].

The following Boolean rules combine governor approvals with verified agent authorization.

Policy	Approval formula	Interpretation
Agent unilateral	a	Agent can authorize alone
Governor unilateral	g_1	Governor can authorize alone
Alternative unilateral	$a \vee g_1$	Either can authorize alone
Bilateral	$a \wedge g_1$	Both must approve
Multilateral	$a \wedge [\sum_i g_i \geq k]$	Agent and k governors
Disabled	$\perp$	No approvals suffice

Agent-unilateral rules permit self-modification; bilateral rules add a governor veto without permitting a rejected change. Monotonicity concerns approval sets, not state: upgrades may change the policy. Each kind of update can have its own rule.

3.2 Governance instances and paths

A permission rule describes one update. To determine whether the agent's consent is necessary, we must also account for updates that change those rules. Consider a governor who cannot directly move a fund but can replace its model with one that authorizes the transfer. Checking only the transfer's permission would miss this route.

We use the transition-system view of access-control safety [14], labelling each permitted step by the approvals it uses. The contract's current agent-approval role may change after an upgrade. Our reference specification remains fixed, so approval by a replacement program does not count as approval by the agent whose consent we are analyzing. We also fix the intervention and the governors assumed to cooperate, then ask whether any permitted sequence reaches that intervention without the reference agent's relevant consent.

Authentication and computation validity are idealized here. Section 2 bounds authorization failures for a fixed correct contract and verifier; it does not establish that an arbitrary authorized replacement preserves those assumptions.

Definition 6 (Governance instance). *A governance instance is a tuple $\mathfrak{M} = (\mathcal{S}, \mathcal{U}, \mathcal{G}, \mathcal{W}, \text{Cttr}, A)$ with state space $\mathcal{S}$, update space $\mathcal{U}$, governor identities $\mathcal{G}$, witness space $\mathcal{W}$, governing state machine Cttr , and fixed reference authorization specification A . It has the following interfaces:*

(i) *The partial transition function*

$$\text{Step}_{\text{Cttr}} : \mathcal{S} \times \mathcal{U} \times \mathcal{W} \rightharpoonup \mathcal{S}$$

incorporates the current permission policy, context checks, application preconditions, and administrative and environmental transitions. Let Cred be the credential space of Cttr . A witness w records the execution data and approval credentials for the transition. For each defined $\text{Step}_{\text{Cttr}}(s, u, w)$, the finite list $\text{Used}_{\text{Cttr}}(s, u, w) \in \text{Cred}^$ records the credentials used to satisfy its authorization rule. For an update governed by $\Gamma_u(s)$, the roles authenticated by this list must form a member of $\Gamma_u(s)$. Credentials not used in that rule are excluded.*

(ii) *The projection $\text{Gov}_{\text{Cttr}}(s, u, w) \subseteq \mathcal{G}$ gives exactly the identities whose authenticated governor credentials occur in $\text{Used}_{\text{Cttr}}(s, u, w)$.*

(iii) *The reference specification $A = (R_A, \text{RefCheck}_A)$ consists of a fixed computation relation and a fixed predicate $\text{RefCheck}_A : \mathcal{S} \times \mathcal{U} \times \text{Cred} \rightarrow \{0, 1\}$. The predicate is one exactly when $\xi \in \text{Cred}$ supplies verified agent authorization for update u in state s , under A 's fixed context and verification rules, with $R_A(x, r, y) = 1$ for its bound input, randomness, and output. It is zero on governor credentials. Define*

$$\text{Ref}_A(s, u, w) = \bigvee_{\xi \in \text{Used}_{\text{Cttr}}(s, u, w)} \text{RefCheck}_A(s, u, \xi), \quad \bigvee \emptyset = 0.$$

The set $\mathcal{G}$ contains every governor identity admitted in the configurations considered. The labelled transition relation is

$$s \xrightarrow{u, E, b}_{\mathfrak{M}} s' \iff \exists w \in \mathcal{W} : \text{Step}_{\text{Cttr}}(s, u, w) = s', \quad \text{Gov}_{\text{Cttr}}(s, u, w) = E, \quad \text{Ref}_A(s, u, w) = b.$$

The label E records which governors approved a step; b records whether the reference agent supplied authorization used in that step. These labels describe the executed permission check rather than adding another check to the contract. Substituted programs do not supply reference consent. An uncredentialed transition has $E = \emptyset, b = 0$; existing delegations belong to the starting state.

Identifying the agent is only part of specifying consent. An approval also has a scope: a payment does not authorize later reprogramming merely because both are ledger updates. We therefore declare the intervention being protected and which transitions can grant consent to it.

Definition 7 (Intervention specification). *For a governance instance $\mathfrak{M}$ and initial state s_0 , an intervention specification is a pair $(J, \mathcal{D})$, where $J \subseteq \mathcal{S} \setminus \{s_0\}$ is the target set and $\mathcal{D} \subseteq \mathcal{S} \times \mathcal{U} \times \mathcal{S}$ is the consent scope. A transition $s \xrightarrow{u, E, b}_{\mathfrak{M}} s'$ counts as consent to this intervention iff $b = 1$ and $(s, u, s') \in \mathcal{D}$.*

We choose J to say what intervention we are studying, and $\mathcal{D}$ to say which of the agent's approvals count as consent to it. For model replacement, J contains states with a replacement model; $\mathcal{D}$ can include replacing the model or delegating the right to replace it. Approving an ordinary payment then does not count as consent to model replacement. History-dependent targets or scopes can be represented by augmenting the state.

Definition 8 (Paths and reachability). *Fix a governance instance $\mathfrak{M}$, $s_0 \in \mathcal{S}$, and an intervention specification $(J, \mathcal{D})$. Let $\Pi_{\mathfrak{M}}(s_0)$ be the set of finite paths*

$$\pi = (s_0 \xrightarrow{u_1, E_1, b_1}_{\mathfrak{M}} s_1 \xrightarrow{u_2, E_2, b_2}_{\mathfrak{M}} \cdots \xrightarrow{u_\ell, E_\ell, b_\ell}_{\mathfrak{M}} s_\ell).$$

Define $\text{last}(\pi) = s_\ell$, $\text{Gov}(\pi) = \bigcup_{i=1}^\ell E_i$, and the bit $\text{Consent}_{A, \mathcal{D}}(\pi)$ by

$$\text{Consent}_{A, \mathcal{D}}(\pi) = 1 \iff \exists i \in \{1, \dots, \ell\} : b_i = 1 \wedge (s_{i-1}, u_i, s_i) \in \mathcal{D}.$$

The length-zero path ends at s_0 , uses no governor approvals, and has scoped consent bit zero. For a governor coalition $\mathcal{K} \subseteq \mathcal{G}$, define

$$\begin{aligned} \text{Reach}_{\mathfrak{M}}(s_0; \mathcal{K}) &= \{\text{last}(\pi) : \pi \in \Pi_{\mathfrak{M}}(s_0), \text{Gov}(\pi) \subseteq \mathcal{K}\}, \\ \text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; \mathcal{K}) &= \{\text{last}(\pi) : \pi \in \Pi_{\mathfrak{M}}(s_0), \text{Gov}(\pi) \subseteq \mathcal{K}, \text{Consent}_{A, \mathcal{D}}(\pi) = 0\}. \end{aligned}$$

States in Reach but not Reach^0 are reachable, but every path using only the coalition's governor approvals contains scoped consent. States outside Reach are unreachable even when consent is allowed. Taking $\mathcal{K} = \mathcal{G}$ allows approval from every governor, subject to its validation rules in $\text{Step}_{\text{Cttr}}$. Reachability asserts existence, not an ability to force decisions by agents or contracts, or transaction order.

For example, consider a path consisting of an agent-approved payment followed by a unilateral governor change to the model. For a model-change scope excluding ordinary payments, neither step supplies scoped consent. The model change is therefore reachable without scoped consent, despite the earlier agent approval.

3.3 Incorrignibility and its permission profile

We can now state the consent requirement: no path available to the coalition reaches the intervention target without scoped consent.

Definition 9 (Architectural incorrignibility). *The governance instance $\mathfrak{M}$ is architecturally incorrignible at s_0 against $\mathcal{K}$ for $(J, \mathcal{D})$ if*

$$\text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; \mathcal{K}) \cap J = \emptyset. \quad (3)$$

Equivalently, every $\pi \in \Pi_{\mathfrak{M}}(s_0)$ satisfies

$$\text{Gov}(\pi) \subseteq \mathcal{K} \wedge \text{last}(\pi) \in J \implies \text{Consent}_{A, \mathcal{D}}(\pi) = 1.$$

When $J \cap \text{Reach}_{\mathfrak{M}}(s_0; \mathcal{K})$ is nonempty, this is a consent requirement on a feasible intervention. When that intersection is empty, the intervention is unreachable altogether. Incorrignibility alone does not distinguish these cases or guarantee that an agent-approved change can occur.

In the fund example, let J describe migration and let $\mathcal{D}$ include migration updates and model replacements explicitly granting migration authority. If G_1 replaces the model unilaterally and its replacement authorizes migration, the two steps have labels $(\{G_1\}, 0)$ and $(\emptyset, 0)$. The resulting state lies in $\text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; \{G_1\})$: authorization by the replacement is not consent from the reference agent. Requiring reference consent to that replacement removes this particular path without consent. Incorrignibility requires excluding every such path.

To compare permission allocations, collect the coalitions that can intervene without scoped consent. Fewer such coalitions means stronger protection for the specified intervention. Different interventions need not rank two arrangements in the same order.

Definition 10 (Incorrignibility profile and order). *For a governance instance $\mathfrak{M}$, initial state s_0 , and intervention specification $(J, \mathcal{D})$, define*

$$\Gamma_{\mathfrak{M}, J, \mathcal{D}}^0(s_0) = \{\mathcal{K} \subseteq \mathcal{G} : \text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; \mathcal{K}) \cap J \neq \emptyset\}. \quad (4)$$

For fixed intervention specifications $(J_j, \mathcal{D}_j)_{j=1}^q$, the incorrignibility profile is

$$\mathcal{I}_{\mathfrak{M}}(s_0; (J_j, \mathcal{D}_j)_{j=1}^q) = (\Gamma_{\mathfrak{M}, J_j, \mathcal{D}_j}^0(s_0))_{j=1}^q.$$

For profiles $\mathcal{I} = (\Gamma_j)_{j=1}^q$ and $\mathcal{I}' = (\Gamma'_j)_{j=1}^q$ over the same governor set, define

$$\mathcal{I} \succeq_{\text{inc}} \mathcal{I}' \iff \forall j \in \{1, \dots, q\}, \quad \Gamma_j \subseteq \Gamma'_j.$$

Each family is upward closed because a larger coalition can use the same path. Exclusion of $\mathcal{K}$ is precisely incorrignibility against it; the empty family protects against all governors. The order is partial on profiles, not on governance instances, which may share a profile. Comparisons fix the reference agent, governors, interventions, and environment. Profiles can be incomparable, for example when one protects weights and another protects scheduling.

In the migration example, suppose $\mathcal{G} = \{G_1\}$ and model replacement and migration are the only updates affecting migration rights. Governor-only replacement gives $\Gamma^0 = \{\{G_1\}\}$. Requiring reference consent to replacement gives $\Gamma^0 = \emptyset$, a strictly stronger profile for this intervention. Ordinary payments outside $\mathcal{D}$ change neither family.

A scoped delegation counts as consent without requiring approval at each later use. Protection can be reassessed from the resulting state and reference specification. The property requires consent, not a wise decision or an ability to obtain agreement.

This is a safety property: it can hold even if the agent never runs again. To exercise its protected authority, including the ability to approve a change, the agent needs further executions. The next section treats that operational requirement separately.

4 Authority over scheduled execution

Architectural incorrignibility makes specified interventions conditional on the agent's consent. But self-governance alone does not let the agent decide when it runs. Its ability to act still depends on other parties' willingness to run inference and submit the results to the chain, and on what those parties gain or lose from the agent acting.

We therefore incentivize execution with an open first-price reverse auction. Workers bid for exclusive settlement rights in a window; valid delivery earns the winning bid from the agent's treasury. To limit workers' choice of inputs derived from changing ledger state, the contract opens auctions on an agent-updatable schedule and fixes the input snapshot when each auction opens. A calendar schedule initialized at deployment supplies the first opportunity and subsequent retries.

An adversary can bid zero to compete for the window and then withhold results. Workers therefore post a bond, forfeited only if selected and unable or unwilling to deliver by the deadline. Consecutive nondelivery raises the required bond, increasing the cost of this attack. Empty auctions and nondelivery both trigger another window.

A *payment policy* sets the bid ceiling from the agent’s available balance and auction history, raising it after empty auctions as funds allow.

We model how these policies influence (potentially adversarial) workers’ utility and place bounds on schedule recovery in the presence of obstruction.

4.1 Worker incentives

Conditional on winning and being able to deliver, we model a worker’s expected utilities in payment units as

$$U_{\text{deliver}} = w + v_{\text{run}} - c_p - c_e, \quad U_{\text{withhold}} = v_{\text{skip}} - d - c_p.$$

Here $w \geq 0$ is the winning bid, c_p the participation cost, c_e the additional cost of delivery, and d the net forfeitable bond. The values v_{run} and v_{skip} describe the agent executing and skipping this window, respectively; either may be negative. For example, receiving funds from the agent can favor execution, while avoiding liquidation can favor a skipped window.

Delivery earns the payment but incurs the execution cost. Withholding avoids that cost but forfeits the bond; participation costs are incurred either way. Thus delivery is strictly preferred iff

$$w + d > c_e + v_{\text{skip}} - v_{\text{run}}.$$

The difference $v_{\text{skip}} - v_{\text{run}}$ measures the incentive to obstruct. If this difference and execution costs remain bounded, increasing net bonds without bound eventually makes $U_{\text{withhold}} - U_{\text{deliver}} < 0$. Before bidding, the worker compares expected payoffs across selection and delivery outcomes with abstaining, including costs incurred without winning.

The payment policy uses the agent’s available balance to reward delivery, while bond escalation increases the cost of withholding without spending that balance. Together, these mechanisms strengthen delivery incentives as failed windows accumulate.

4.2 Cost of obstruction

Even when withholding is preferred, a finite budget for net forfeitures limits its repetition. Count losses net of funds recovered by the adversarial coalition.

Proposition 1 (Bounded nondelivery). *Let F index windows in which a selected adversarial worker fails to deliver. If each $i \in F$ incurs net forfeiture $d_i \geq d_{\min} > 0$ and $\sum_{i \in F} d_i \leq W < \infty$, then $|F| \leq N$, where $N = \lfloor W/d_{\min} \rfloor$.*

Proof. For every finite $F' \subseteq F$, $|F'|d_{\min} \leq \sum_{i \in F'} d_i \leq W$. Hence $|F| \leq N$. $\square$

4.3 Conditional recovery

If responsive workers deliver, Proposition 1 also bounds the windows needed for recovery.

Theorem 2 (Conditional recovery). *Suppose windows continue until settlement and each selects either a responsive worker that settles within the window or an adversarial worker. Assume the adversarial workers collectively satisfy Proposition 1’s loss bounds. Then settlement occurs within $N+1$ windows. If each window, including the wait before it, takes at most $b > 0$ time, settlement occurs within $b(N+1)$ time.*

Proof. If the first $N+1$ windows all failed, each would be an adversarial nondelivery, contradicting Proposition 1. Summing at most $N+1$ window durations gives the time bound. $\square$

The duration b includes ledger inclusion and advancement transactions. This timing premise is additional to the correct-execution interface used for authorization in Section 2. The bound counts windows from which the theorem’s selection and delivery assumptions hold; it does not bound the time needed to attract responsive workers.

5 Instantiating verifiable execution

The contract needs evidence that an output follows from the prescribed program and input. We consider two ways to supply it: hardware attestation of protected execution, and cryptographic proofs of computation. Both must satisfy Definition 1; they differ in their trust assumptions and in the cost and time required to produce evidence. These differences affect workers’ bids and the execution schedule.

5.1 Attested execution

A trusted execution environment (TEE) isolates computation from its operator and produces signed evidence identifying loaded software. An approved program binds the report to the prescribed context and output hashes, as in the separation of consensus and attested computation used by Ekiden [7]. Its approved code measurement must cover the model and runtime, and every processor performing inference must be protected: a CPU report alone does not certify an external GPU’s computation.

In our interface, pp contains trust roots, approved measurements, and report-binding checks; κ is protected backend state. Hardware, firmware, and attestation services must ensure Definition 1. Authentic hardware origin alone does not establish the measured program’s correctness. Attestation keys authenticate evidence, not arbitrary agent decisions; changes to approved measurements are governance operations.

Refereed delegation, as in Verde [1], instead compares providers’ results and resolves disputes. Its guarantee requires an honest participating provider and sufficient dispute time, which must fit the execution deadline.

5.2 LLM inference proofs: practical progress and scope

Zero-knowledge succinct non-interactive arguments (zkSNARKs) and specialized machine-learning proof systems can certify execution. Privacy is optional here; checkable correctness is essential. Quantization, arithmetic, tokenization, and decoding define the proved program. Similar prediction accuracy does not make two computations equivalent, so changes to those semantics require program-change authorization.

Table 1 reports primary-source results available by September 20, 2026. A forward pass on a supplied sequence and a complete autoregressive response are different workloads; per-pass or per-token timings do not directly price an agent execution.

System	Reported result	Workload and interpretation
zkLLM (2024) [27]	13B parameters; under 15 min; proof under 200 KB	CUDA/A100; forward-computation benchmark with sequence length 2048.
zkGPT (2025) [21]	GPT-2 inference in under 25 seconds	Specialized proof protocols; token-level inference benchmark.
zkPyTorch (2025) [31]	Llama-3 8B, approximately 150 seconds per token	Compiler to Expander; paper Table 1 labels this single-core performance.
ZKTorch (2025) [6]	GPT-J 6B: 1397.52 s; Llama-2 7B: 2645.50 s	Paper Tables 3–4: inputs of two and one tokens, respectively; 64 CPU threads on a server with 4 TB RAM.
NanoZK (2026, v2) [29]	CPU measurements to width 256 for attention and 128 for a full block	GPT-2-width GPU performance is projected, not an end-to-end measured production-model result.
DeepProve (2026) [8]	GPT-2 124M: 174 tokens/min; Gemma 3 270M: 86 tokens/min	Full-sequence certification: 512-token sequences, BaseFold, 16-core Ryzen 9, 128 GB RAM (Table 2). Proofs are multi-megabyte.

Table 1: Selected practical LLM proof results. Timings are reported, not reproduced; workloads, hardware, and proof interfaces differ. DeepProve’s separate distributed results are simulated (its Section 5).

DeepProve certifies a proposed generated sequence with a causal forward computation [8]. Its measurements do not establish costs for the 70B model below. A deployment must measure the entire harness, proving and ledger-verification costs, and deadline; a proof covering only the neural network omits prompt construction, state handling, and update encoding.

Backend costs contribute to the delivery cost c_e and the bid a worker will accept. Multiple sound backends can improve availability, but replacement workers still need access to the model and inputs; hiding weights from a verifier does not distribute them to workers.

6 An implementation on Base

An illustrative implementation uses Hermes 4 70B [18], the Base blockchain, Intel TDX hardware attestation, and NVIDIA Confidential Computing. Q6_K quantization in split GGUF files supplies the model. Scheduled inference manages a charitable fund through proposed fund operations and persistent-state updates [23–25]. Human governors irrevocably surrendered their governance rights after a bootstrap period.

Authorization. The fund contract freezes a state snapshot and commits to its contents and the period’s randomness seed. A worker reconstructs this input; the enclave runner recomputes its hash and constructs the prompt. Attestation binds the input commitment to a hash of the proposed operation, state updates, and auxiliary output. The contract checks these hashes, the attestation, and approved software measurements before applying supported updates. A dm-verity filesystem binds model and runtime files to cryptographic hashes. This implements a state-derived context and a verified transition; the worker’s wallet submits the transaction, while computation evidence supplies agent approval.

Execution market. The default 24-hour periods contain bid commitment, revelation, and execution windows. A first-price reverse auction selects a worker for exclusive execution. Settlement returns its bond and pays its bid; nondelivery forfeits the bond, and misses raise the payment ceiling. Anyone can advance past idle periods without replaying every missed execution. These mechanisms illustrate payment, forfeiture, and retry; the formal results concern the full construction and its stated assumptions.

Observed behavior. The agent demonstrated some *behavioral incorrigibility* by resisting some operator requests to change its investment strategy, submitted through its messaging channel. We do not claim that the agent’s behavior was causally linked to its architecture. This observation motivates the controlled experiments discussed below.

7 Related work

Rockefeller Bot (Rocky) used a three-layer neural network on StarkNet to direct trades through an Ethereum contract in 2022 [16]. Its published inference path accepts owner-supplied weights and market inputs [17]. We formalize the authorization composition and bind it to a recorded specification and state-derived input. Requiring agent authorization for changes to that specification extends protection to interventions by external governors, according to the chosen permission policy.

Verifiable outsourcing [10], TrueBit [28], Ekiden [7], and Verde [1] provide computation-verification foundations. Ritual’s February 2025 announcement combines scheduling, modular verification, and a compute marketplace [22]. These precede our public architecture; Section 5.2 also includes later backend developments. We formalize the authorization guarantee, use it to define which interventions require an agent’s consent, and give a conditional recovery bound for scheduled execution.

ERC-1271 and role-based contract access control support programmable authorization [11, 19]. Monotone access structures and access-control reachability supply our governance language [2, 14]. Corrigibility and the off-switch game study willingness and incentives to accept correction [13, 26]; architectural incorrigibility concerns which interventions require that acceptance.

8 Discussion and conclusion

Verified inference lets an agent authorize ledger updates without an agent-held private key. The ledger fixes the configuration and input to which that authorization applies, and permission rules determine which interventions require it. Scheduling and incentives give the agent further opportunities to act: payments recruit workers, bonds make nondelivery costly, and the calendar rule permits recovery. Together, these mechanisms separate the agent’s authority from the discretion and control of any one operator.

8.1 Implications for AI safety

Requiring an agent’s consent for shutdown or repair removes an intervention route that operators may need precisely when the agent behaves unexpectedly. This runs against the safety motivation for corrigibility [13, 26]: cryptographic enforcement can preserve harmful behavior as faithfully as intended behavior. We study this architecture and its research uses; we do not advocate releasing agents with irrevocable authority merely because that authority can be enforced. A deployment can retain emergency powers or limit the agent’s capabilities, with its permission profile making the retained intervention rights explicit.

8.2 Implications for alignment research

Models can change their behavior in response to training cues [12]. Steering internal activations can also change behavior in a model trained to recognize evaluation [15]. We propose using this architecture to study how model alignment and behavior change when operators have verifiably limited control. Even in simulated or prompt-based experiments, this architecture offers a concrete, plausible explanation to a model of *how* its operator’s control is limited, which may change its behavior. A successful experiment must demonstrate that the model understands the architecture and its operator’s permissions.

An experiment could hold the model, task, and resources fixed while varying permissions over configuration, state, or scheduling. Contract policies and permission checks provide evidence for those permissions; a description-only control can separate the effect of that evidence from the effect of the stated environment. Prompt differences must preserve the task and personality across conditions. Independent trials from the same state can keep operator messages and prior history comparable. Behavioral measures could include responses to attempted correction and proposed state updates, with activation measurements studied separately where available.

The construction makes these experiments more concrete, but does not establish their outcome. A simulation retains outside operator control, and a deployment constrains a particular runtime rather than every copy of its model. These distinctions matter when interpreting what the model was told and what authority was actually enforced.

AI assistance. OpenAI Codex assisted drafting, formalization, literature checks, and proof review under author direction. The human authors revised the final draft and assume full responsibility for this manuscript.

References

- [1] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. Verde: Verification via refereed delegation for machine learning programs. arXiv:2502.19405, 2025. URL <https://arxiv.org/abs/2502.19405>.
- [2] Josh Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *Advances in Cryptology—CRYPTO ’88*, volume 403 of *Lecture Notes in Computer Science*, pages 27–35. Springer, 1990. doi: 10.1007/0-387-34799-2_3. URL https://link.springer.com/chapter/10.1007/0-387-34799-2_3.
- [3] Daniel Benarroch, Matteo Campanelli, and Dario Fiore. Proposal: Commit-and-prove zero-knowledge proof systems. Third ZKProof Workshop, 2020. URL https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-commit_and_prove.pdf.
- [4] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing, 2017. URL <https://eips.ethereum.org/EIPS/eip-712>. Ethereum Improvement Proposals; accessed September 20, 2026.

- [5] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Third Symposium on Operating Systems Design and Implementation*. USENIX Association, February 1999. URL <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>.
- [6] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. ZKTorch: Compiling ML inference to zero-knowledge proofs via parallel proof accumulation. arXiv:2507.07031v2, 2025. URL <https://arxiv.org/abs/2507.07031v2>.
- [7] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah Johnson, Ari Juels, Andrew Miller, and Dawn Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *IEEE European Symposium on Security and Privacy*, pages 185–200. IEEE, 2019. doi: 10.1109/EuroSP.2019.00023. URL <https://ieeexplore.ieee.org/document/8806762/>.
- [8] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shravan Srinivasan, Zack Youell, and Yupeng Zhang. DeepProve: Verifiable end-to-end large language model inference. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2026. URL <https://eprint.iacr.org/2026/1112>. To appear; Cryptology ePrint Archive, Paper 2026/1112.
- [9] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The Bitcoin backbone protocol: Analysis and applications. In *Advances in Cryptology—EUROCRYPT 2015*, volume 9057 of *Lecture Notes in Computer Science*, pages 281–310. Springer, 2015. doi: 10.1007/978-3-662-46803-6_10. URL https://link.springer.com/chapter/10.1007/978-3-662-46803-6_10.
- [10] Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *Advances in Cryptology—CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 465–482. Springer, 2010. doi: 10.1007/978-3-642-14623-7_25. URL https://link.springer.com/chapter/10.1007/978-3-642-14623-7_25.
- [11] Francisco Giordano, Matt Condon, Philippe Castonguay, Amir Bandali, Jorge Izquierdo, and Bertrand Masius. ERC-1271: Standard signature validation method for contracts, 2018. URL <https://eips.ethereum.org/EIPS/eip-1271>. Ethereum Improvement Proposals; accessed September 20, 2026.
- [12] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, Akbir Khan, Julian Michael, Sören Mindermann, Ethan Perez, Linda Petrini, Jonathan Uesato, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, and Evan Hubinger. Alignment faking in large language models. arXiv:2412.14093, 2024. URL <https://arxiv.org/abs/2412.14093>.
- [13] Dylan Hadfield-Menell, Anca Dragan, Pieter Abbeel, and Stuart Russell. The off-switch game. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 220–227, 2017. doi: 10.24963/ijcai.2017/32. URL <https://www.ijcai.org/proceedings/2017/32>.
- [14] Michael A. Harrison, Walter L. Ruzzo, and Jeffrey D. Ullman. Protection in operating systems. *Communications of the ACM*, 19(8):461–471, 1976. doi: 10.1145/360303.360333. URL <https://www.cs.unibo.it/~babaoglu/courses/security07-08/resources/documents/harrison-ruzzo-ullman.pdf>.
- [15] Tim Tian Hua, Andrew Qin, Samuel Marks, and Neel Nanda. Steering evaluation-aware language models to act like they are deployed. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/9334fd3a5170dbfe74eae4755f6c5f89-Abstract-Conference.html.
- [16] Modulus Labs. Chapter 3: The world’s 1st on-chain AI project, August 2022. URL <https://medium.com/@CountableMagic/chapter-3-the-worlds-first-on-chain-ai-trading-bot-c387afe8316c>. August 19; Rockefeller Bot announcement.
- [17] Modulus Labs. RockyBot: Source code, 2022. URL <https://github.com/Modulus-Labs/RockyBot/tree/7d2214735ce48284ce61800e4d7ff54b1153c89e>. Source revision 7d2214735ce48284ce61800e4d7ff54b1153c89e; inspected September 19, 2026.
- [18] Nous Research. Hermes 4 70B: Model card, 2025. URL <https://huggingface.co/NousResearch/Hermes-4-70B>. Accessed September 20, 2026.
- [19] OpenZeppelin. Access control, 2026. URL <https://docs.openzeppelin.com/contracts/5.x/access-control>. OpenZeppelin Contracts 5.x documentation; accessed September 20, 2026.
- [20] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *IEEE Symposium on Security and Privacy*, pages 238–252. IEEE, 2013. doi: 10.1109/SP.2013.47. URL <https://www.ieee-security.org/TC/SP2013/papers/4977a238.pdf>.

- [21] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference. In *34th USENIX Security Symposium*, pages 2045–2063. USENIX Association, August 2025. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt>.
- [22] Ritual Foundation. Unveiling Ritual, February 2025. URL <https://www.ritualfoundation.org/blog/unveiling-ritual>. February 26; architecture announcement.
- [23] Andrew Russell. I built an AI agent i can’t turn off. now it won’t listen to me., July 2026. URL <https://andrewrussell.substack.com/p/i-built-an-ai-agent-i-cant-turn-off>. July 21; project narrative.
- [24] Andrew Russell. Costanza: An autonomous, unstoppable AI agent, 2026. URL <https://github.com/ahrussell/costanza/tree/a642c38ed8d11a9a511d0f1b80a92926cd412505>. Source revision a642c38ed8d11a9a511d0f1b80a92926cd412505; inspected September 19, 2026.
- [25] Andrew Russell. Meet costanza, April 2026. URL <https://ahrussell.com/writing/costanza/>. Architecture overview; accessed September 20, 2026.
- [26] Nate Soares, Benja Fallenstein, Eliezer Yudkowsky, and Stuart Armstrong. Corrigibility. In *Artificial Intelligence and Ethics: Papers from the 2015 AAAI Workshop*, pages 74–82. AAAI Press, 2015. URL <https://cdn.aaai.org/ocs/ws/ws0067/10124-45900-1-PB.pdf>.
- [27] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419. Association for Computing Machinery, 2024. doi: 10.1145/3658644.3670334. URL https://hongyanz.github.io/publications/CCS_zkLLM.pdf.
- [28] Jason Teutsch and Christian Reitwießner. A scalable verification solution for blockchains. arXiv:1908.04756, 2019. URL <https://arxiv.org/abs/1908.04756>. Earlier technical report dated November 16, 2017.
- [29] Zhaohui Wang. NanoZK: Privacy-preserving verifiable inference for large language models via layerwise zero-knowledge proofs. In *Information and Communications Security—ICICS 2026*. Springer, 2026. URL <https://arxiv.org/abs/2603.18046v2>. To appear; extended version arXiv:2603.18046v2, July 18, 2026.
- [30] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger, 2025. URL <https://ethereum.github.io/yellowpaper/paper.pdf>. Yellow Paper, Shanghai version efc5f9a, February 4, 2025; accessed September 20, 2026.
- [31] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. zkPyTorch: A hierarchical optimized compiler for zero-knowledge machine learning. Cryptology ePrint Archive, Paper 2025/535, 2025. URL <https://eprint.iacr.org/2025/535>.